

gowkhtmltopdf - Architecture

A work-alike for *wkhtmltopdf* built for controlled reports: invoices, statements, tables, multi-page documents with headers/footers, TOCs and PDF outlines. Everything from **load** to **write** runs inside the Go binary.

DOCS

11 deep-dives in
documentation/architecture/

DOMAINS

10 sections on this page

SCALE

~71 600 lines of Go

COMMIT

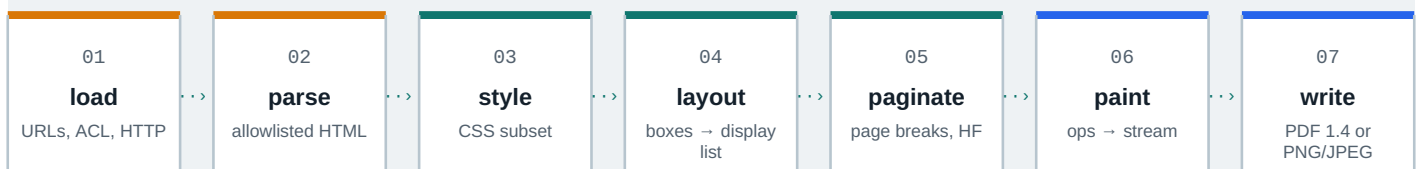
ef526f9 2026-08-11

INPUTS

SHARED ENGINE

OUTPUTS

Conversion pipeline one document, seven stages



Reading order jump to any domain

- 01 · [Entrypoints & CLI](#)
- 02 · [Public library API](#)
- 03 · [Settings system](#)
- 04 · [Load layer](#)
- 05 · [HTML parser](#)
- 06 · [CSS subsystem](#)
- 07 · [Layout engine](#)
- 08 · [Convert pipeline](#)
- 09 · [PDF writer](#)
- 10 · [Image output & SVG](#)
- 11 · [Dependency DAG](#)
- 12 · [PDF vs image](#)
- 13 · [Security](#)

01

Entrypoints & CLI

Three entry points: two CLIs and the library; every flag funnels through dotted settings into the `convert.Request` seam.

The outermost shell turns raw process arguments into a settings-shaped `cli.Command` and hands it to the application adapters that build engine requests. One parser serves both binaries through a `Mode` bitmask, and the parser is the only place that understands the *wkhtmltopdf*-compatible multi-object grammar: `page`, `cover` and `toc` objects plus free positionals.

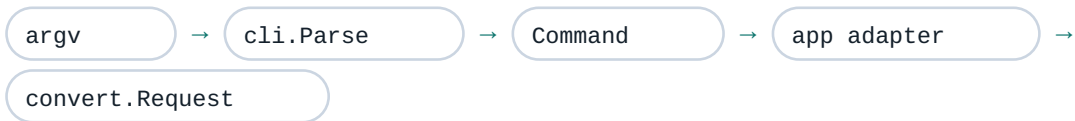
ONE PARSER, TWO BINARIES - ARGV IN, REQUEST OUT

```
$ gowkhtmltopdf --page-size A4 cover cover.html toc page ch1.html out.pdf
```

```
write PDF: 3 pages · outline depth 2 · exit 0
```

CODE	MEANING	SOURCE
0	ok	success
1	error	parse or engine failure
2	HTTP 404	settings.HttpStatusError
3	HTTP 401	settings.HttpStatusError

Flags land in settings through dotted names: `--page-size` writes `size.pagesize`; free positionals become `Command.Objects` and the last one becomes `Output`. Errors surface as `gowkhtmltopdf: unknown option --dpi` on stderr with exit code 1, while HTTP failures report `2` or `3`. Long option names may split at word boundaries when printed.



cmd/gowkhtmltopdf PDF binary main - parse (ModePDF), doc-flag dispatch, signals, exit code; 83 thin lines
cmd/gowkhtmltoimage Image binary main - parse (ModelImage), no TOC-XSL dump, exit code; 53 thin lines
internal/cli Parser core: Command, Parse, Mode bitmask, ~71 registered flags, ExitCode mapping
internal/app Adapters BuildPDFRequest / RunImage turn Command into engine requests

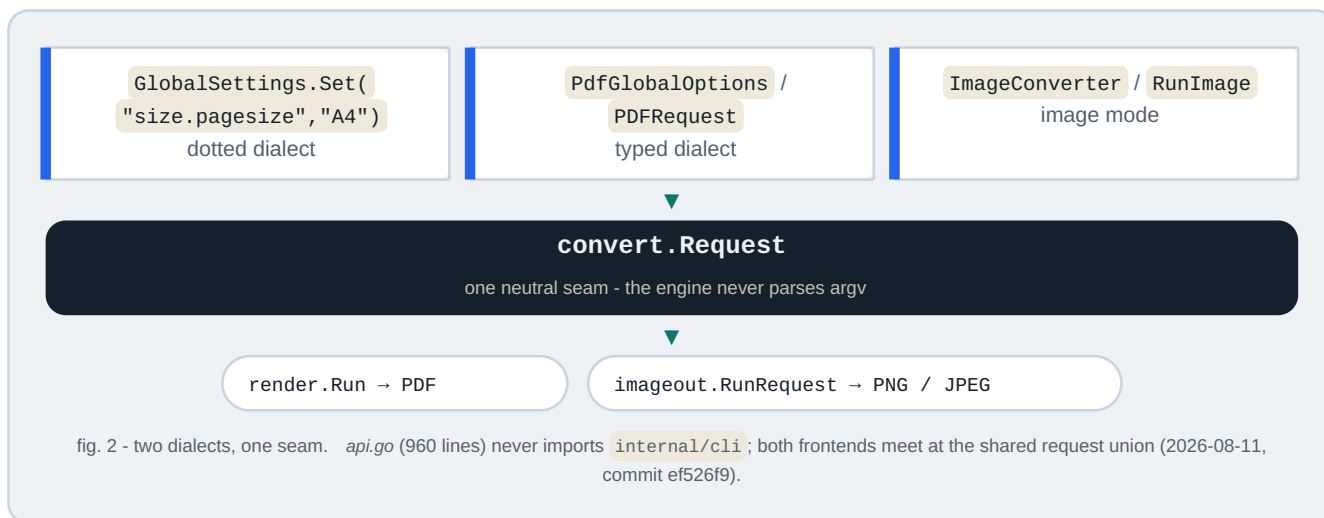
- `cli.Command` - parse result: Global/Image settings, Objects, Output / OutputWriter
- `cli.Parse(argv, Mode)` - token loop, object grammar, pending remapping
- `cli.ExitCode(err)` - 0/1/2/3 via HttpStatusCode
- `settings.Set("size.pagesize", ...)` - the dotted settings home appliers write
- `newFreshObject()` - TOC objects never consume pending - no ghost page

Security: local-file ACL is opt-in (`--enable-local-file-access` dual-writes `load.blocklocalfileaccess`); Policy A rejects inert wkhtmltopdf stubs (`--dpi`, `--javascript-delay`) as unknown rather than no-ops; validation precedes output truncation; the CLI spawns no subprocess and cancels cleanly on SIGINT/SIGTERM via a context.

Position: `argv` → `settings` → `Request` (left edge of the pipeline) · see [documentation/architecture/01-entrypoints-cli.md](#)

Converter / ImageConverter public API: typed and dotted dialects, deep-copy ownership, in-memory output.

The root package `gowkhtmltopdf` is the idiomatic, `stdlib`-only Go surface for the engine. It configures a conversion with either `wkhtmltopdf`-compatible dotted strings or compile-time-discoverable typed builders, and drives the pipeline: `Converter.Convert(ctx)` reduces to an internal `convert.Request` handed to `convert.Run` (PDF) or `imageout.RunRequest` (image), with all output captured in memory. This package is also the module boundary that enforces the product rule - `pure Go, no cgo, stdlib + internal only` - so any new external dependency would have to cross its import list first.



`api.go`

`api_test.go`

`doc.go`

`internal/convert,`
`settings, imageout, line,`
`errs`

all exported types: settings wrappers, converters, typed requests, hooks, sentinels
round trips, deep-copy mutation isolation, cancellation, callbacks, PNG/JPEG decode
quick start, dotted settings, ACL pair, in-memory source kind, thread-safety contract
the five internal dependencies the root package adapts

- `Converter` + `ImageConverter` - lifecycle: `New`, `AddObject`, `Convert(ctx)`, `Output()`
- `GlobalSettings` / `ObjectSettings` - dotted `Set` / `Get` via the reflect key tables
- `PDFRequest` / `ImageRequest` + `RunPDF` / `RunImage` - typed one-shot; `Now func() time.Time` gives byte-stable output
- `ConvertHTML` - one-shot helper: clone global, `SetBody(html, base)`, convert, output
- `Version()` - banner `0.12.7-dev (gowkhtmltopdf pure-go)`, tracking `LibraryVersion 0.12.7-dev`
- Sentinels - `ErrNoPageObjectsAdded`, `ErrEmptyHTML`, `ErrNilContext`, matchable with `errors.Is`
- `convertHooks` + `lineLog` - engine log lines classified by `line.SeverityOf` into `OnInfo` / `OnWarn` / `OnError`

Surface	Contract
<code>Converter</code>	PDF, multi-object, <code>OnPhase</code> / <code>OnProgress</code> (final 100); no <code>Now</code> hook
<code>ImageConverter</code>	PNG/JPEG, single most-recent page, lazy zero-value init, no phase callbacks
<code>RunPDF</code> / <code>RunImage</code>	typed path - <code>Now</code> injectable for reproducible metadata

Security posture: local-file ACL is default-deny and needs the same two-step opt-in as the CLI (`enablelocalfileaccess=true` + `load.blocklocalfileaccess=false`); `SetBody` / `ConvertHTML` bypass URL guessing so in-memory HTML cannot be coerced into SSRF; context cancellation threads through every load. Limits: no `HttpError` on `Converter` (CLI-only today), no TOC as a first-class object,

one image page only. Every boundary copies: `SetBody` , `AddObject` , `Output` .

Same seam as the CLIs: `api.go` → `convertHooks` → `Request` → `render.Run` - deep-dive:

`documentation/architecture/02-library-api.md`

03 Settings system & errors

wkhtmltopdf-style settings model with reflection key tables; settings and errs are leaf packages.

`internal/settings` is the single source of truth for every user-facing knob - page geometry, margins, headers/footers, TOC behaviour, load policy, image output - and the only package that speaks the wkhtmltopdf dotted-name vocabulary (`margin.top` , `load.jsdelay` , `web.background`). It sits upstream of everything: `argv` and library calls funnel through `Set("margin.top", "25mm")` , and every engine package reads the typed structs it default-initialises. Its companion leaf `internal/errs` owns the shared sentinel errors across the library boundary.

DOTTED-KEY PATH - ONE KNOB, TYPED ALL THE WAY DOWN

`--margin-top 25mm` → `Set("margin.top", "25mm")` → `Margin.Top = 25 mm` →
`10 mm` → `28.346 pt`

Booleans coerce liberally (`"` / `true` / `1` / `yes` / `on`); unknown keys fail loudly - a typo like `margin.topx` errors instead of degrading.

POLICY A SETTINGS HONESTY

Only options with an engine consumer get typed fields. Inert wkhtml keys (`dpi` , `javascript` , `js-delay` , `produce-forms` , `default-encoding` , `.`) sink into an `Ignored` map: *accepted* so existing invocations do not hard-fail, *inert* so they are never silently promoted to typed stubs. Origin: the 2026-08-06 ponytail review, phase 1.



accepted-inert wkhtml
keys, round-trippable via
`Get`



ISO/ANSI page sizes -
`A4 = 595.28 × 841.89 pt`

`internal/settings/settings.go`

Typed model: `PdfGlobal` , `PdfObject` , `ImageGlobal` plus `Web` , `LoadGlobal` , `LoadPage` , `HeaderFooter` , `TableOfContent` - wkhtmltopdf-compatible defaults (A4 portrait, 10 mm margins, depth-4 outline).

`internal/settings/reflect.go`

Descriptor engine: dotted `Set` / `Get` , type-coercing setters, ignored-key tables - built once at init, zero per-call reflection cost.

`internal/settings/unitreal.go`

`UnitReal` (`10mm` , `1.5in` , `12pt` , `96px` , `100%`), the 23-entry page-size table, and `HttpStatusError` (`404` → `2` , `401` → `3` , else `1`).

· `pagesize.go` · `httperror.go`

`internal/errs/errs.go`

Shared sentinels: `ErrNilContext` , `ErrNilLoader` , `ErrNilCommand` , `ErrNilRequest` - re-exported by `api.go` and `internal/app` .

- `PdfGlobal` / `PdfObject` / `ImageGlobal` - the three settings payloads carried by `cli.Command` and `convert.Request` .
- `Set` / `Get` - dotted-key surface; accepted inert keys round-trip, unknown keys error.

- `UnitReal` - unit-suffix scalar; `Points()` / `Mm()` conversion.
- `ParsePageSize` — case-insensitive ISO/ANSI names to points; empty string → A4.
- `HttpErrorCode` — 404 → 2, 401 → 3, else 1; consumed by `cli.ExitCode` via an interface check.

SECURITY & LIMITS

Local file access is denied by default (`BlockLocalFileAccess: true`); `--allow` prefixes are the only ACL widening; proxies must be absolute `http(s)` URLs. Inert-key acceptance is a compat surface, not an attack surface - the engine never executes scripts or creates forms.

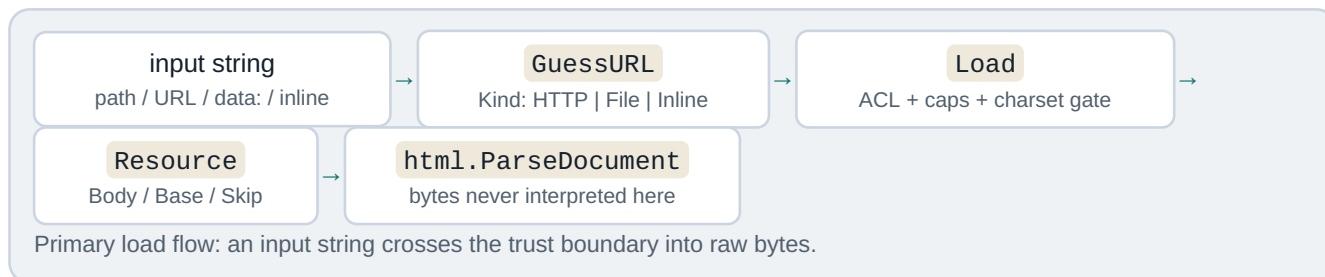
Known gaps: `default-encoding` is accepted-ignored; `size.pagesize` dual-write is transitional; object-level `web.background` is inert; the typed builder covers global PDF options only.

Position: upstream of every entry point — dotted Set/Get → `UnitReal` → page sizes → convert pageGeometry. Source: [documentation/architecture/03-settings.md](#) (commit ef526f9, 2026-08-11).

04 Load layer - URLs, I/O & ACL

Resource-fetching seam and trust boundary: URL guessing, local-file ACL, HTTP hardening, size caps.

`internal/load` owns the first pipeline stage and the two resource seams the engine depends on: the **primary load** (one `Loader.Load` per body object, cover, TOC, or header/footer) and the **subresource seam** (`ResourceContext.Fetch` for linked CSS, images and `@font-face` URLs). It is the lowest internal package - only `internal/settings` and the Go standard library sit beneath it - and it carries the product security posture: default-deny local-file ACL, connect/response timeouts, a 10-redirect cap, a 100 MiB body cap, and a strict UTF-8/ASCII charset gate.



Primary load

`Loader.Load(ctx, input, pageLoad)` resolves pages, covers, TOCs and header/footer HTML like top-level documents. Budgets: 30 s connect, 60 s response default, 100 MiB cap, max 10 redirects, per-page `--timeout` override.

Subresource seam

`ResourceContext.Fetch(ref)` binds a document base plus the per-page policy, so CSS, images and fonts inherit the same ACL and caps - e.g. `internal/convert/prepare/styles.go` collects linked stylesheets through this one narrow seam.

[internal/load/load.go](#) URL guessing, HTTP/file/data: fetching, cookie jar, proxy, auth, POST, ACL, caps, charset gate (1,322 lines)

[internal/load/load_test.go](#) 30 behavioural tests: ACL matrix, body caps, timeouts, redirects, subresources, concurrency

[internal/load/doc.go](#) Package doc: "reimplements the MultiPageLoader orchestration layer"

- `Kind` - resolved input class: HTTP | File | Inline
- `Resource` - fetched document: URL, Base, Body, Skip

- `ResourceContext` - the one sanctioned subresource seam
- `AccessController` - default-deny ACL, symlink-resolved prefixes
- `NewLoaderWithError` - fail-fast constructor used at the request boundary

Global enable	Object block	Read allowed
off	on (default)	no
off	off	no
on	on	no
on	off	yes

URL shapes handled: `http://host/path`, `file://localhost/tmp/report.html` (remote file hosts refused), `data:` URIs, and bare input defaulting to `http://<input>` - a direct port of `wkhtmltopdf`'s `guessUrlFromString`.

Trust boundary owner: every input string crosses here into raw bytes; nothing below `internal/load` touches the network or filesystem.

Security: default-deny ACL with `--allow` prefix expansion; no JavaScript ever; oversized bodies are rejected, never truncated; the TOCTOU window between ACL check and read is accepted residual risk; HTTP egress to `localhost/RFC1918` is intentionally unrestricted (input-trust model). Full model: `documentation/THREAT-MODEL.md`.

First pipeline stage feeding `internal/html` + the `prepare` subresource seam · deep-dive: `documentation/architecture/04-load.md`

05 HTML parser: tokenizer & tree

Generic tokenizer and tree builder: entities, tolerant parsing, script content stripped at layout.

`internal/html` is the parse stage of the pipeline: a hand-rolled, *stdlib-only* leaf that owns the **only DOM in the repo** - every downstream stage walks its `Node` tree. Scanning is streaming: `scanTokens` emits tokens through a sink callback while the tree is built incrementally in the same pass, so a whole-token slice is never materialized.

Token constructs the scanner recognizes

text run
`<!-- comment -->`
`<!doctype>`
`<?PI?>`
`</end>`
`<start>`
`<raw>`

skipped
verbatim

Raw-text elements - `script`, `style`, `textarea`, `title` - are captured verbatim up to their real closing tag, then hidden by the UA sheet (`display:none`, `style_values.go:986`): script content is stripped at *layout*, not at parse. Entities decode once, at text-append and attribute application, via `UnescapeEntities` over `stdlib.html.UnescapeString`.

A ruby annotation parses as one inline run - base, parens and `reading` are ordinary inline nodes (CJK ruby needs `--font-path` for glyphs):

XML([Extensible Markup Language](#)) 2026-08-11 · `html.go` ≈ 832 lines · `allowlistedbyconstruction`

Tolerance is a feature: browser-grade-recovery is deliberately out of scope, while malformed input degrades to a usable tree- six sentinel errors, never a panic.

[internal/html/html.go](#) Tokenizer, tree builder, Node model, walkers, element vocabularies (≈832 lines)

[internal/html/entities.go](#) UnescapeEntities - &-gated stdlib entity decoding

[internal/html/html_test.go](#) Same-package tests: tokenizer, tree, malformed-input table (≈901 lines)

Consumers css.Match, layout style resolution, outline heading collection, convert HF/TOC/islands

- `Parse` / `ParseDocument` - string vs bytes (+BOM) entry points
- `Node.Walk` - pre-order traversal used by every consumer
- `scanTokens` - streaming scanner with sink callback
- `autoClose` - implicit-close vocabulary (li, p, tr, td, th.)
- `mergeRootElement` - html/head/body merge instead of nesting
- `uaDecIs` - UA sheet hiding script/style/title/head/meta/link

Security: no script execution by construction - event-handler attributes are inert data; charset is enforced (UTF-8/ASCII only) at the load seam before parsing; hostile input cannot panic.

Position: load - [html](#) - (css matching, layout) · deep-dive: [documentation/architecture/05-html-parser.md](#)

06

CSS subsystem: parse, selectors & cascade

CSS subset: selectors, cascade, media and container queries; unknown rules degrade; `:has` and `:target` supported.

`internal/css` is a pure parsing, matching and value library that sits between the HTML tree and the layout engine. It owns the *CSS subset* the converter accepts and reports matches and specificity to `internal/layout`, which performs the cascade. It never fetches resources, never resolves page size, and never decides layout - the contract is quoted in its package doc (*css.go:1-18*).

parse-time

match-time

value helpers

never panics

html

css

layout

pdf / image

Supported surface: `type`, `.class`, `#id`, attribute selectors, `:first-child` / `:last-child` / `:nth-child`, `:has()`, `:not()`, descendant/child/sibling combinators, `@media`, `@container`, `!important`, and inline `style=""`. Everything else degrades without panicking- and unsupported selectors never silently degrade to the host element (`writePseudoLiteral` keeps unknown pseudos on the compound, so `li:target` can never paint every list item).

Degrade rules - how the subset stays honest

- `garbage` is dropped: only unbalanced braces error, and `ParseNeverPanics` pins the no-panic property on adversarial input.
- `never-match` pseudos (`hover`, `active`, `focus`, `target`) stay on the compound and match `false` - a static PDF has no pointer state.
- `skip` at-rules: `@import`, `@supports`, `@layer`, `@keyframes`, nesting are silently skipped.
- `parse-early`: media preludes stay raw strings (the viewport is only known at conversion); `:nth-child` arguments compile to integer form so matching is pure arithmetic.

Known gaps - what degrades today

WIRE FORMAT - ONE DECLARATION BLOCK

```
--accent : #b4532a ; custom property
color : var(--accent, #b4532a ) ; var() with fallback
font-size : 1.1rem ; !important single global tier
```

Unit math is tuned for IEEE float cancellation: 25.4mm is computed as $val \times 72 \div 25.4$ so it cancels cleanly to 72pt instead of 71.999.; px maps at 0.75 (96css px /in $\rightarrow$ 72 pt /in); the media em base is 12 pt. Specificity is Selectors-4: :has() / :not() contribute the max of their arguments - a triple (a, b, c) ¹ with a = ids, b = classes/attrs/pseudos, c = types.

¹ Functional pseudos contribute the specificity of their most specific argument, not a flat class-level count.

internal/css/css.go	Sheet/rule/selector model, parser, at-rules, right-to-left matching, specificity (~1784 lines)
internal/css/values.go	Inline style, !important, length/color/font parsing, custom-property resolution
internal/css/container.go	@container prelude, boolean condition tree, LengthToPt
internal/css/has.go	:has() / :not() combinator walk; @media evaluation
media.go	

- css.Parse - whole-sheet parse; only unbalanced braces error
- css.Match / MatchPseudo - selector vs node, right-to-left; ::before/::after shapes
- css.Specificity - (a,b,c) triple; :has()/:not() max-of-arguments
- css.MediaMatches - raw @media prelude vs conversion viewport
- css.ResolveCustomProps - the single place custom-property policy lives
- css.HasContainerRules - fast gate; layout skips its second style pass when false
- css.ParseSelectors - strict list parse for --exclude-from-outline

Import rule: internal/css $\rightarrow$ internal/html $\rightarrow$ stdlib only; css never imports layout, convert, pdf, load, or settings - viewport/media/container context is always passed as arguments. **Security:** css executes no code and fetches nothing; url() is honored only in @font-face src via the loader's ACL, and stylesheet rule budgets are capped in preparation (soft warn 25 000, hard cap 1 000 000).

Position: html - css - layout style resolution · measured 2026-08 · deep-dive: documentation/architecture/06-css.md

07

Layout engine & line breaking

Style cascade plus all formatting contexts, line breaking, pagination, emitting the display list.

internal/layout is the largest domain in the repo: it runs the cascade and builds an absolute-positioned *display list* of drawing ops on a continuous y-down canvas. Layout also owns pagination - page-break policies, thead repetition, sticky clamps, and the op-split machinery. One engine feeds three painters: PDF, header/footer bands, and the PNG/JPEG rasterizer.



layout and paginate both live in internal/layout; the same ops feed pdf.Paint, PaintBand (HTML header/footer) and the imageout rasterizer.

One display-list Op OP KINDS

Every op carries a stable ID. Kinds: OpFillRect, OpStrokeRect, OpLine, OpText, OpImage, OpLinkURI, OpBullet. Rects that split across a page boundary keep the source ID, so element-to-op ownership survives fragmentation.

Final z-order is resolved once by `PaintOrder` and shared by all three backends.

Formatting contexts (report-lite subset)

- block & inline flow
- flex lite (row / column)
- multicol lite
- relative / absolute / fixed lite
- tables: colspan, rowspan, thead repeat
- grid lite + subgrid-inherit
- float lite + clear
- print sticky, 2D transforms

Key entry points (layout)

- `LayoutContext` / `WithWorkspace` - the layout passes
- `PaintContext` - paginate and paint into a `pdf.Document`
- `PaintBandContext` - single clipped band for HTML header/footer
- `CloneResult` - deep copy for the TOC page-count fixpoint

Scale: ~33.6k lines incl. tests, 27 production files, 62 test files, 244 test functions.

internal/layout

cascade, boxes, ops, pagination, paint; entry points `LayoutContext` / `PaintContext`

internal/line

naming note: log-line severity protocol (`line.Emit`), NOT text breaking - line breaking lives in layout `inline.go` / `inline_collect.go`

internal/svg

Rasterize is used for inline SVG images; layout stays one layer below convert and never imports `convert/load/settings`

- `Result` - Ops, Pages `[]int`, Locations `[]ElementLocation`
- `ResolvedStyle` - ~1.3 KB per style, interned per run via `styleStore`
- `ElementLocation` - feeds `outline.Lookup` and `#frag` link assembly
- `Options` - `Images func(src)` callback keeps the ACL-gated loader injected
- `Workspace.Release` - display-list storage reuse for the page-islands path

Images arrive only through the injected `Images` callback (ACL-gated); layout never opens URLs and never runs JS. Work is bounded: the pagination fixpoint caps at 10 iterations, and SVG rasterizes at a 1024 px cap. Lite limits: positioning, sticky, float, flex/grid/multicol are report subsets; overflow is not general clipping; no vertical writing modes; HTML header/footer bands clip rather than paginate.

style - boxes - paginate - display list (consumed by pdf OR imageout) · deep-dive: [documentation/architecture/07-layout.md](https://github.com/robertkrimen/otto/blob/master/doc/architecture/07-layout.md)

08

Convert pipeline - job orchestration

Orchestration hub: the `Request` union, 3-stage `render.Pipeline`, headers/footers, TOC fixpoint, outline, page islands.

`internal/convert` is the only package that coordinates a complete conversion. It owns the neutral `Request` union (`convert.go:51`), runs the per-object render loop, and drives every document-wide pass - TOC, outline, links, headers/footers, copies - before writing a single PDF. The lifecycle is delegated to a three-stage `render.Pipeline`, keeping PDF specifics out of the stage driver.

1 RenderObjects

per object: prepare → layout →
paint



Assemble

TOC → outline → links → doc
→ copies → HF



Finalize

doc.Write(output sink)

Lifecycle adapter: render.Pipeline (render/pipeline.go) owns stage order and cancellation; pdfPipeline (pdf_pipeline.go) owns the PDF detail. Cancellation is checked at every object boundary and between stages.

Assemble, in order

1. `assembleTOC` - two-pass fixed point (`renderTOCObjects` , `toc.go:221`): `iteration 1` measures with `tocGuess 0`, `iteration 2` rennumbers with the measured total and keeps the final layouts; pages reorder so the TOC comes first.
2. `assembleOutline` - pure heading tree (`internal/outline` , no PDF types) → `emitOutline` → `pdf.Outline` with final page refs.
3. `assembleLinks` - TOC forward/back links plus `#id` GoTo annotations (`applyInternalLinks`).
4. `assembleDocument` - page plan, title, compression, grayscale, creation time.
5. `assembleCopies` - collate / non-collate materialization via `render.Plan` .
6. `assembleHeadersFooters` - final pass; needs [topage] and section/subsection, which only exist once the whole document is final.

Fact: headers/footers are painted once at the very end - after copies - because `[topage]` and section/subsection are only known when the whole document exists. Auto margins reserve the bands at layout time (`effectiveMargins` , `hf.go:596`); the draw pass clips ink into those same bands.

internal/convert

job model, per-object loop, document-wide passes

internal/convert/prepare

shared load → parse → sheets → fonts phase (same seam used by image mode)

internal/convert/render

Pipeline lifecycle driver + Plan page-index model

internal/outline

pure heading tree, dump XML, section lookup (no layout/PDF result types)

- `Request` - neutral pipeline input: Global, Image, Objects, Now, Output, OutlineOutput (`convert.go:51`)
- `render.Pipeline` - RenderObjects / Assemble / Finalize (`render/pipeline.go:12`)
- `runContext` - one conversion lifecycle: loader, fonts, single pdf.Document
- `objectState` - per-object geometry, HF, headings, navigation projection
- `pagePlan` - owner map + Remap after TOC reorder / copies (`page_plan.go`)
- `drawHeadersFootersResult` - final HF pass with real page numbers (`hf.go:637`)

Security: the loader (proxy/ACL boundary) is built at the top of `Run` before fonts or layout state; every subresource fetch inherits its ACL, timeouts and body caps. Header/footer values that look like HTML are treated as raw markup, never as URLs - blocking a local-path injection vector. Output sinks are caller-supplied writers; convert never opens paths. Budgets: 10 000 objects, 1 000 copies, 100 000 pages.

Limits: `--xsl-style-sheet` is not implemented (built-in TOC template, warning + fallback); the TOC fixed point is bounded to two iterations; HTML headers/footers are single-band clamps.

Progress is a (phase, percent) callback:  100 = progressComplete at Finalize; the CLI prints phase lines unless `--quiet` .

09

PDF 1.4 writer

PDF 1.4 sink: Flate streams, font subsetting, Type0/CID, xref, outlines, byte-stable output.

internal/pdf is the lowermost writer layer: it turns the layout display list into a viewer-openable PDF 1.4 file using only the Go standard library plus one allowlisted shaping module (go-text/typesetting). It owns the document object model and serialization, the content-stream operator language, the font pipeline (parse → subset → simple or Type0/CID embed), image embedding under hard caps, and outline/annotation wiring. Image mode reuses its fonts, shaping and glyph outlines but rasterizes instead of writing PDF bytes.

display list → paint → content streams → xref

Font pipeline: untrusted TTF / WOFF1 in, one subset per font per document out



Content-stream language

Every operator is emitted by the **Content** builder: graphics state `q/Q`, paths `m/l/re/f/s`, transforms `cm`, and text `BT/ET/Tf/Tj`, with fonts and images registered into per-page `/Resources` at finalize. Text emits through `TextShow`: pure ASCII fast path, otherwise shape then split mixed Latin / CJK runs.

Serialization

writeTo streams objects through a countingWriter so every xref offset is exact; all `/FlateDecode` streams use RFC 1950 zlib (raw DEFLATE made pages render empty). A typical tail:

```

0000000000 65535 f
0000000015 00000 n
0000000091 00000 n
trailer << /Size 3 /Root 1 0 R >>

```

Subsets are keyed by `fontCache[ v{0|1} | {fingerprint} | {baseName} | {runes} ]` so one deterministic subset per font per document is guaranteed, and hard caps bound untrusted input: **32 MiB** reconstructed WOFF, **16 384 px** image dimension, **16 Mpix** pixel budget.

internal/pdf	Document model, operators, fonts, subsets, images, outlines (≈ 9.1k lines)
internal/pdf/assets	//go:embed Liberation + DejaVu faces; every accessor returns an isolated bytes.Clone
internal/convert	Caller: paints into Document, assembles TOC/outline/links/copies, then doc.Write
internal/imageout	Reuses fonts + shaping + glyph outlines; rasterizes instead of writing PDF bytes

- **Document** - object model; `AddPage`, `ReorderPages`, `DuplicatePage`, `SetOutline`, `Write`
- **Content.TextShow** - ASCII fast path, else shape + split mixed runs; records runes for the subsetter
- **ParseTTF** - sfnt validation; rejects CFF/OTTO and WOFF2 (Brotli not allowlisted)
- **ensureFont** - dispatch simple vs Type0; one subset per font per document (cache key above)
- **emitType0** - Identity-H CIDFontType2; CIDToGIDMap kept uncompressed for viewer safety
- **finalize** - ordered wiring: outlines before catalog, pages tree, then xref + trailer

Security: fonts are untrusted parse input — WOFF1 caps (tables ≤ 1024, per-table ≤ 16 MiB, overlap rejection), bounds-checked tables, no remote https @font-face fetch, system fonts opt-in only. **Determinism:** byte-identical output for identical input; CreationDate is injectable via Request.now() and otherwise pins a fixed date.

display list → paint → content streams → xref (right edge, PDF mode) · deep-dive: documentation/architecture/09-pdf-writer.md

10

Image output & SVG raster

Image-mode tail: rasterize the shared display list to PNG/JPEG; bitmap fonts; SVG via tdewolff/canvas.

Image mode shares the whole *load* → *parse* → *style* → *layout* front half with PDF mode, then diverges at the paint/write tail: `internal/imageout` paints the layout **display list** into an in-memory `image.NRGBA` canvas and encodes a single PNG or JPEG. `internal/svg` rasterizes `` into PNG bytes through the allowlisted `tdewolff/canvas` path - the project's only third-party raster call.

display list → rasterize 2× NRGBA → box-filter → encode PNG/JPEG

Shared upstream, divergent sink - the Aug 2026 consolidation keeps text metrics, glyph forms and paint order on one table with PDF mode.

PDF mode

`internal/pdf` - Flate streams, font subsetting, xref

Image mode

`internal/imageout` - 2× supersample, no hinting, single page

`internal/imageout`

~3,000 lines: render options, supersampled rasterization, paint dispatch, decode/scale caches, PNG/JPEG encode

`internal/svg`

450 lines: `svg.Rasterize` via `tdewolff/canvas` only; `viewBox/width/height` sizing, panic containment

`internal/convert/render`

Shared `RenderObjects` → `Assemble` → `Finalize` lifecycle; `Assemble` is a no-op for image mode

`internal/pdf`

Consumed, not owned: `ShapeRun` text shaping, `GlyphContours`, `DefaultFont`, `ScanFontDirs`

- `RenderContext(ctx, root, opts)` — the core library render: layout result → rasterize → crop
- `RunRequest(ctx, req, log)` - engine seam shared by the `gowkhtmltoimage` CLI and `ImageConverter`
- `rasterizeContext` - paints `layout.Op` ops in `PaintOrder`, box-filters `rasterSS` supersample down
- `ttfDrawString` — TTF outline → bitmap glyphs (no FreeType, no hinting); fractional baseline, fake-bold pass
- `svg.Rasterize(data, maxSide)` — SVG → PNG bytes at intrinsic size; failure is a clean "no image"

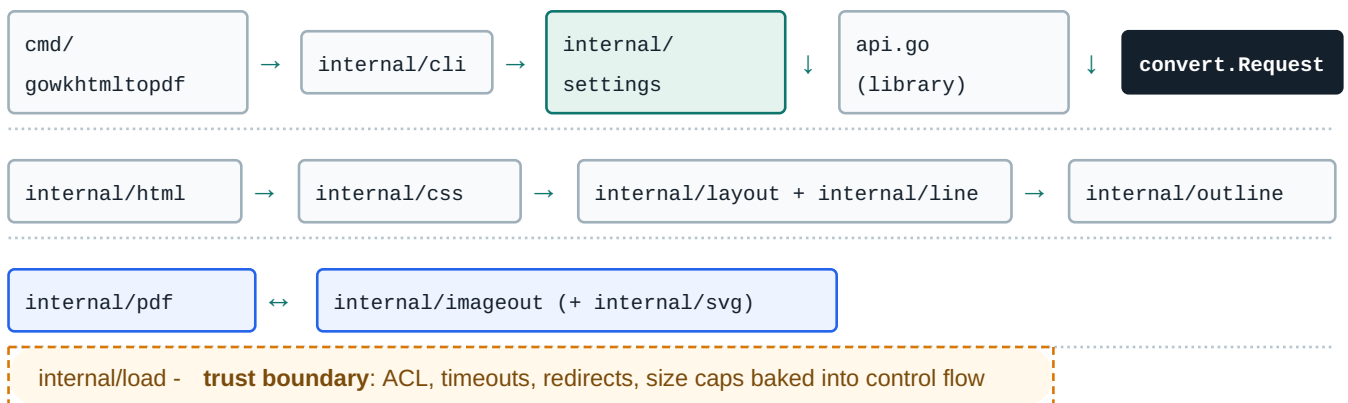
Raster text uses **TTF outline AA** - **AA** via 2× supersampling with a box-filter downscale; no hinting. Fidelity claims are pinned in `10-imageout-svg.md` and `fidelity.md`; the shape was consolidated 2026-08-11.

Security & limits

- ACL merged (`Image.Load` ⊕ `Global.Load`); subresource fetches capped (64 fetches / 32 MiB)
- Decode hardening: ≤32 MiB encoded, dims ≤16,384, ≤16M pixels; canvas ≤64M px / ≤256 MiB
- `--transparent` honored for PNG only; JPEG composites onto white with a warning
- Malformed SVG → clean error, never a crash (`defer recover()` in `rasterizeCanvas`)
- Single-page only: extra page objects and TOC are ignored with a warning
- Nearest-neighbour `<img>` scaling (Go 1.26 removed `image/draw` scalers)

Dependency DAG a strict downward graph toward the trust boundary

Nothing points back up. `internal/cli` is a leaf; the engine never parses argv. Library, image binary and PDF binary funnel into one neutral seam: `convert.Request`. `internal/load` sits at the bottom as the trust boundary - every subresource crosses the same two seams and inherits the same ACL, timeout and size caps.



- 1
- 2 `convert.Request` `render.Pipeline`
- 3 `cli.Command` `is`
- 4 `load.Loader.Load` `load.ResourceContext.Fetch`
- 5

PDF vs image mode same front half, different tail

internal/pdf - PDF 1.4

- content streams + zlib Flate, xref/trailer
- embedded Liberation Sans subsets, /Widths 1000-unit em
- Type0/CID for CJK, WOFF handling
- Catalog outlines, link annotations, metadata
- byte-stable output when `Now` is injected

internal/imageout - PNG/JPEG

- rasterize the shared display list - Assemble is a no-op
- bitmap font rasterizer (2× supersample, no hinting)
- single page; `--transparent` fill-alpha divergence
- SVG-as-image via `tdewolff/canvas` (only third-party raster call)
- reuses pdf shaping + outline data

Security architecture default-deny, enforced at the seams

ACL

Local file access **denied by default**; opt-in via `--enable-local-file-access` / `--allow` prefixes, symlinks

HTTP

30 s connect / 60 s response timeouts, max 10 redirects, 100 MiB body cap on both sides.

JS

No JavaScript, ever - script content stripped at the layout stage.

resolved.

BUDGETS

Rule and resource budgets enforced at the convert layer, not scattered in callers.

HF

Header/footer raw-markup rejected at the convert layer.

Full model: `documentation/THREAT-MODEL.md` · embedding guidance: `documentation/integration-security.md`.

Generated from `documentation/architecture/` - commit `ef526f9` (2026-08-11). Ten domain sections authored by ten parallel subagents; shell composed by the orchestrator. Pure HTML + CSS, no scripts.

MIT License · Copyright © 2026 Chinmay Sawant · [back to top ↑](#)